6 实验结果

本实验成功实现了以下功能：

1. 直线绘制：实现了中点画线法和 Bresenham 画线法，支持任意斜率的直线绘制，通过分 8 个象限处理实现了完整的直线绘制功能
2. 圆绘制：实现了中点画圆法和 Bresenham 画圆法，利用八对称性只需计算 1/8 圆弧即可得到完整的圆，大大提高了绘制效率
3. 线型控制：支持实线、虚线、点线、点划线四种线型，通过 Direct2D 的 StrokeStyle 实现
4. 线宽控制：使用扫描线填充法实现粗线和粗圆的绘制，粗线转换为矩形多边形填充，粗圆转换为环形填充
5. 区域填充：实现了种子填充算法和扫描线填充算法，种子填充适合任意形状的封闭区域，扫描线填充适合已知顶点的多边形
6. **Bezier** 曲线：使用 de Casteljau 递推算法绘制任意阶数的 Bezier 曲线，通过对参数 t 进行采样得到曲线上的点

7 实验总结

通过本次实验，深入理解了计算机图形学中基本图元生成算法的原理和实现方法：

1. 中点画线法和 **Bresenham** 算法：两种算法本质相同，都是通过整数运算避免浮点计算，提高了绘制效率。中点法从几何角度推导，Bresenham 法从误差累积角度推导。
2. 圆的绘制：利用圆的八对称性，只需计算 1/8 圆弧即可得到完整的圆。中点画圆法和 Bresenham 画圆法的决策参数不同，但都能高效地绘制圆。
3. 区域填充：种子填充算法适合填充任意形状的封闭区域，但需要访问像素数据；扫描线填充算法适合已知顶点的多边形，可以利用 GPU 加速。
4. **Bezier** 曲线：de Casteljau 算法具有数值稳定性好、几何意义清晰的优点，通过递推计算可以得到曲线上任意参数对应的点。
5. 粗线绘制：将线段转换为矩形多边形，使用扫描线填充法实现，这种方法可以绘制任意宽度的线段。

本实验加深了对计算机图形学基础算法的理解，提高了使用 Direct2D 进行图形编程的能力。